Documento de Requisitos

Proyecto final – AsistenteIA

Información	Descripción
Universidad	Universidad de San Buenaventura
Programa	Tecnología de Desarrollo de Software – VI semestre
Asignatura	Programación Avanzada y Aplicada
Docente	Sandra Esperanza Sánchez Barragán
Proyecto	AsistenteIA – Asistente académico móvil con IA (.NET MAUI, API REST, SQLite)
Integrantes	Juan Sebastián Garzón Suárez y Nicolás Quiroz

1. Introducción

AsistenteIA es una aplicación móvil académica. El usuario escribe una consulta o elige una plantilla de prompt, la aplicación valida el texto, lo envía mediante una solicitud POST con JSON a una API compatible con Chat Completions (Pollinations), muestra la respuesta y permite guardarla en un historial local con SQLite.

1.1 Objetivo general

Construir, probar y sustentar una aplicación móvil que envíe instrucciones a un modelo de IA mediante una API REST pública, muestre la respuesta, guarde el historial local y aplique criterios de seguridad, privacidad, accesibilidad y uso responsable.

1.2 Objetivos específicos

- Implementar navegación Shell, XAML adaptable y el patrón MVVM con inyección de dependencias.
- Consumir una API REST con POST y JSON de forma asíncrona, con cancelación y manejo de errores.
- Persistir el historial con SQLite (crear, consultar, actualizar y eliminar).
- Proteger la credencial con SecureStorage y aplicar controles de uso responsable de la IA.

1.3 Alcance

Aplicación con cinco pantallas (Inicio, Consultar IA, Historial, Detalle, y Configuración con Acerca de), probada en Windows. Queda fuera del alcance un servidor propio (backend) y la cuenta de usuarios.

2. Requisitos funcionales

Código	Requisito	Componente
RF01	El usuario puede ingresar una pregunta o instrucción.	ChatPage (Editor)

Código	Requisito	Componente
RF02	La aplicación valida que el prompt tenga contenido y una longitud permitida (5 a 2000 caracteres).	AiService.AskAsync
RF03	La aplicación envía una solicitud POST a la API REST de IA.	AiService
RF04	Se muestran estados de carga, respuesta, error y ausencia de conexión.	ChatViewModel (StatusMessage, IsBusy)
RF05	Cada consulta y respuesta puede guardarse en SQLite.	Botón Guardar, DatabaseService
RF06	El usuario consulta, filtra, ve el detalle y elimina elementos del historial.	HistoryPage y DetailPage
RF07	La aplicación ofrece plantillas de prompts académicos editables.	PromptTemplates, Picker
RF08	La configuración permite registrar o actualizar la clave sin mostrarla en pantalla.	SettingsPage (Entry con IsPassword)
RF09	La sección Acerca de explica finalidad, autoría, fuentes, privacidad y límites.	SettingsPage
RF10	La aplicación funciona al menos en Windows o Android y se prueba en el destino elegido.	Plan de pruebas P01–P10

3. Requisitos no funcionales

Código	Categoría	Requisito
RNF01	Seguridad	La clave se almacena solo con SecureStorage; no hay claves en el código, el repositorio ni las capturas.
RNF02	Privacidad	Aviso de no ingresar datos personales; el historial permanece únicamente en el dispositivo.
RNF03	Rendimiento	Las llamadas son asíncronas (la interfaz no se bloquea) y el HttpClient tiene un tiempo máximo de 60 segundos.
RNF04	Usabilidad y accesibilidad	Controles con SemanticProperties, textos legibles y contenedores con desplazamiento para pantallas pequeñas.
RNF05	Mantenibilidad	Separación en Models, Services, ViewModels y Views; dependencias inyectadas.
RNF06	Portabilidad	Modelo y endpoint configurables para poder cambiar de proveedor.
RNF07	Fiabilidad	La aplicación no se cierra ante fallos de red, tiempo agotado o clave inválida.

4. Historias de usuario y criterios de aceptación

ID	Historia de usuario	Criterio de aceptación
HU01	Como estudiante quiero escribir una pregunta para obtener una explicación.	Con prompt y clave válidos se muestra la respuesta y el estado final.
HU02	Como estudiante quiero usar plantillas para formular mejor mis consultas.	Al elegir una plantilla el texto se carga y se puede editar.

ID	Historia de usuario	Criterio de aceptación
HU03	Como estudiante quiero guardar respuestas para repasarlas después.	Al pulsar Guardar el registro aparece en el historial y persiste tras reiniciar.
HU04	Como estudiante quiero buscar y eliminar consultas antiguas.	La búsqueda filtra la lista y eliminar pide confirmación.
HU05	Como estudiante quiero marcar una respuesta como verificada tras contrastarla.	El detalle muestra y conserva el estado de verificación.
HU06	Como usuario quiero configurar mi clave sin que se muestre.	La clave se oculta, se guarda cifrada y puede eliminarse con confirmación.
HU07	Como usuario quiero conocer los límites y la privacidad de la aplicación.	La sección Acerca de explica finalidad, autoría, fuentes, privacidad y límites.
HU08	Como usuario quiero entender los errores que ocurran.	Ante falta de red, clave inválida o tiempo agotado se muestra un mensaje comprensible.

5. Flujo principal

1. El usuario abre la pestaña Consultar, elige una plantilla (opcional) y escribe su consulta.
2. La aplicación valida la longitud, comprueba la conectividad y recupera la clave de SecureStorage.
3. Se envía un POST a `chat/completions` con el modelo configurado.
4. Se muestra la respuesta junto con el aviso de verificación.
5. El usuario guarda la consulta; aparece en Historial, donde puede abrir el detalle, marcarla como verificada o eliminarla.

6. Restricciones

- La aplicación móvil no debe incluir claves secretas escritas en el código ni en archivos de configuración publicados.
- El proveedor de IA puede modificar modelos, cuotas o autenticación; por eso el modelo y el endpoint son configurables.
- Equipo de 2 estudiantes y cinco sprints de construcción.